

# Program Analysis

Roland Leißa (based on slides by S. Hack)

University of Göttingen

## Data Flow Analysis

- Dead Code Elimination and Liveness Analysis

- Definite Assignment of Variables

## Lattices and Fixpoints of Monotone Functions

- Algorithms

## Constant Propagation

- Unreachable Code

- Domain Cooperation

## Abstract Interpretation

## Pentagons and Bound Checking

## Global Value Numbering

# Data Flow Analysis

---

# Dead Code Elimination

## Goal

Eliminate computations whose results are never used.

## Definition

A variable  $x$  is **live** at a program point  $p$  if there is a path from  $p$  to a use of  $x$  that does not contain a definition. A variable is **dead** at  $p$  if it is not live at  $p$ . An instruction defining a dead variable is **dead code**.

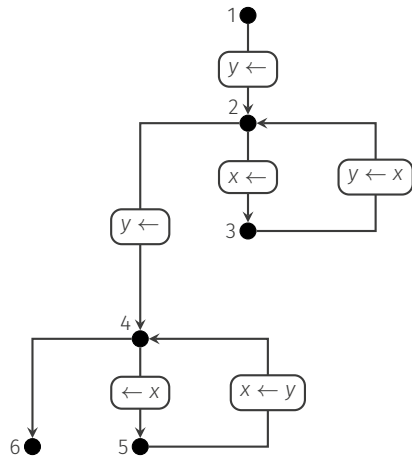
## Soundness

We **must not** eliminate computations whose results are live. Hence, every over-approximation of liveness is **safe** with respect to eliminating dead code.

## Problem

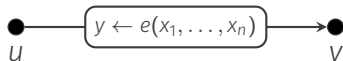
How do we compute the set of live variables for each program point?

# Example



## A Local Approach

Consider an edge in the CFG

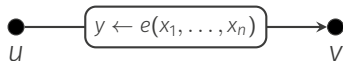


Assume we know the set  $L_v$  of variables live at  $v$ .

What can we say about  $L_u$  based on this information?

## A Local Approach

Consider an edge in the CFG



Assume we know the set  $L_v$  of variables live at  $v$ .

What can we say about  $L_u$  based on this information?

$$L_u \supseteq (L_v \setminus \{y\}) \cup \{x_1, \dots, x_n\}$$

I.e. for each edge  $(u, v)$  in the CFG there is a function

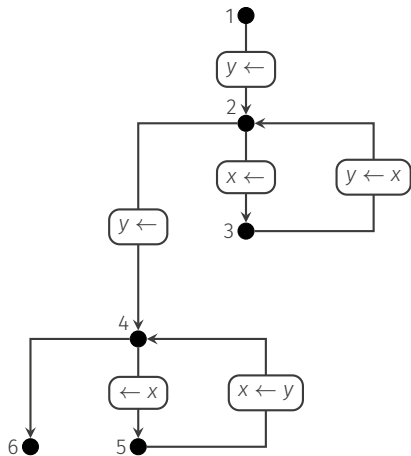
$$\text{lv}_{uv}(L) \triangleq (L \setminus \text{def}_{uv}) \cup \text{use}_{uv}$$

that computes the “effects” with respect to liveness analysis.

**Idea**

Create (in-) equation system from CFG.

# Example





# The Inequation System

Our inequation system has an equation for each edge:

$$L_u \supseteq lv_{uv}(L_v) \quad \text{for each edge } (u, v) \in E$$

Let us bring this into a form where we have one equation per program point:

$$L_u \supseteq \bigcup_{v \in \text{succ}(u)} lv_{uv}(L_v) \quad \text{for each node } u \in V$$

Note that  $a \cup b$  is the smallest set such that  $a \cup b \supseteq a$  and  $a \cup b \supseteq b$ .

Using a function

$$\mathbb{L} \triangleq V \rightarrow \mathcal{P}(\text{Vars}), \quad LV : \mathbb{L} \rightarrow \mathbb{L} \quad LV(\mathbf{L}) \triangleq \lambda u. \bigcup_{v \in \text{succ}(u)} lv_{uv}(\mathbf{L}(v))$$

we can turn this into a single inequation  $\mathbf{L} \supseteq LV(\mathbf{L})$

## Conclusion

Every solution  $\mathbf{L}$  is an over-approximation to the set of live variables.

# Another Analysis: Definite Assignment

## Goal

Check that on each path to a use of a variable, that the variable has been defined before the use.

(This is a mandatory analysis in Java and used by C/C++ compilers when warnings are enabled.)

## Definition

A variable  $x$  is definitely assigned at  $p$  if each path from  $r$  to  $p$  contains a definition of  $x$ .

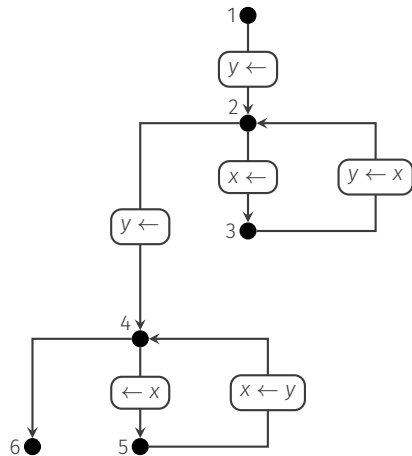
## Soundness

Every non-definitely assigned variable must be reported.

Warning about a definitely assigned variable is tolerable.

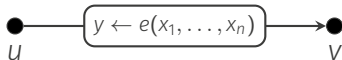
Hence, every **under-approximation** on the definitely assigned vars leads to sound warnings

# Example



# Definite Assignment

Same game. Consider an edge:



- The set of definitely assigned variables  $A_v$  based on  $A_u$ :  $A_v \subseteq A_u \cup \{y\}$
- So again, we have a function and an inequation per edge:

$$A_v \subseteq \text{da}_{uv}(A_u) \quad \text{for each edge } (u, v) \in E$$

- Turning this into an equation system with one inequation per node:

$$A_v \subseteq \bigcap_{u \in \text{pred}(v)} \text{da}_{uv}(A_u) \quad \text{for each node } v \in V$$

$\Rightarrow$

$$\mathbf{A} \subseteq \text{DA}(\mathbf{A}) \quad \text{with } \mathbf{A} \in \mathbb{A}, \quad \text{DA} : \mathbb{A} \rightarrow \mathbb{A}, \quad \mathbb{A} \triangleq V \rightarrow \mathcal{P}(\text{Vars})$$

## Summary so far

So far, data flow analyses have the following ingredients:

- An analysis domain (Information that is derived about the program)  
(liveness, definite Assignment:  $V \rightarrow \mathcal{P}(\text{Vars})$ )
- Transfer functions on edges
- A join operator that is compatible with the order of the inequation  
(liveness:  $\cup$ , definite assignment:  $\cap$ )
- Using both, we can build the “big single” function (LV, AD)

### Open Question

How do we compute a solution to the inequation system?

To this end, we will look into lattices, monotone functions, and their fixpoints.

# Lattices and Fixpoints of Monotone Functions

---

# Partial Orders

Let  $P$  be a set. A binary relation  $\leq$  on  $P$  is a **partial order** iff it is:

1. reflexive:  $(\forall x \in P) x \leq x$
2. transitive:  $(\forall x, y, z \in P) x \leq y \wedge y \leq z \implies x \leq z$
3. antisymmetric:  $(\forall x, y \in P) x \leq y \wedge y \leq x \implies x = y$

The pair  $(P, \leq)$  is called **ordered set**.

An element  $\perp$  with  $\perp \leq x$  for all  $x \in P$  is called **bottom** element. It is unique. Analogously,  $\top$  is called **top** element, if  $\top \geq x$  for all  $x \in P$ .

Let  $P$  be an ordered set. The **dual**  $P^D$  of  $P$  is obtained by defining  $x \leq y$  in  $P^D$  whenever  $y \leq x$  in  $P$ .

For every statement  $\Phi$  about  $P$  there is a dual statement  $\Phi^D$  about  $P^D$ . It is obtained from  $P$  by exchanging  $\leq$  by  $\geq$ .

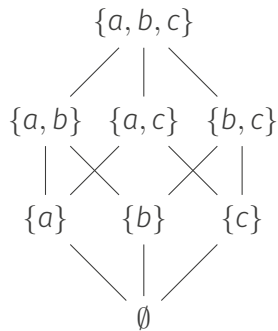
If  $\Phi$  is true for all ordered sets,  $\Phi^D$  is also true for all ordered sets.



# Hasse Diagrams

A partial order  $(P, \leq)$  is typically visualized by a Hasse diagram:

- Elements of  $P$  are points in the plane
- If  $x \leq z$ , then  $z$  is drawn above  $x$ .
- If  $x \leq z$ , and there is no  $y$  with  $x \leq y \leq z$ , then  $x$  and  $z$  are connected by a line



$$P = \mathcal{P}(\{a, b, c\})$$

The Hasse diagram of the dual of  $P$  is obtained by “turning”  $P$ ’s by  $180^\circ$

# Upper and Lower Bounds

Let  $(P, \leq)$  be a partial ordered set and let  $S \subseteq P$ . An element  $x \in P$  is a **lower bound** of  $S$ , if  $x \leq s$  for all  $s \in S$ . Let

$$S^\ell = \{x \in P \mid (\forall s \in S) x \leq s\}$$

be the set of all lower bounds of the set  $S$ . **Dually:**

$$S^u = \{x \in P \mid (\forall s \in S) x \geq s\}$$

Note:  $\emptyset^u = \emptyset^\ell = P$ .

If  $S^\ell$  has a greatest element, it is called the **greatest lower bound (meet)** and is written  $\bigwedge S$ . (Dually for **least upper bound (join)** and  $\bigvee S$ .)

Let  $P$  be an ordered set.

## Definition

- If  $\bigvee\{x, y\}$  and  $\bigwedge\{x, y\}$  exist for every pair  $x, y \in P$  then  $P$  is called a **lattice**.

We also write  $x \vee y$  for  $\bigvee\{x, y\}$  (and  $x \wedge y$  for  $\bigwedge\{x, y\}$ ).

- If for every  $S \subseteq P$ ,  $\bigvee S$  and  $\bigwedge S$  exist, then  $P$  is called a **complete lattice**.

## Lemma

*Let  $L$  be a lattice and let  $a, b \in L$ . The following statements are equivalent:*

1.  $a \leq b$
2.  $a \wedge b = a$
3.  $a \vee b = b$

# Conditions for Completeness

## Theorem

Let  $(P, \leq)$  be an ordered set. The following statements are equivalent:

1.  $P$  is a complete lattice.
2. for every subset  $S \subseteq P$ ,  $\bigwedge S$  exists.
3. for every non-empty subset  $S \subseteq P$ ,  $\bigwedge S$  exists and  $P$  has a  $\top$

## Definition (ACC)

An ordered set  $P$  fulfills the ascending chain condition (ACC), if for every sequence

$$x_1 \leq \cdots x_n \leq \cdots \quad x_i \in P$$

there exists  $k \in \mathbb{N}$  such that  $x_k = x_{k+1} = x_{k+2} = \cdots$ .

## Theorem

Let  $P$  be a lattice.  $P$  is complete if  $P$  has a  $\perp$  and satisfies (ACC).

## Finite (non-empty) Lattices Are Complete

Associativity allows us to write sequences of joins unambiguously without brackets. One can show (by induction) that

$$\bigvee \{a_1, \dots, a_n\} = a_1 \vee \dots \vee a_n$$

for  $\{a_1, \dots, a_n\} \in L$ ,  $n \geq 2$ . Thus, for any **finite, non-empty** subset  $F \in L$ ,  $\bigvee$  and  $\bigwedge$  exist.

Thus, every finite lattice is **bounded** (has a greatest and least element) with

$$\top = \bigvee L \quad \perp = \bigwedge L$$

Finally, because finite lattices have  $\perp$  ( $\top$ ), it exists  $\bigvee \emptyset$  ( $\bigwedge \emptyset$ ):

$$\perp = \bigvee \emptyset \quad \top = \bigwedge \emptyset$$

Hence, finite lattices are **complete**.

Let  $(P, \leq)$  be an ordered set/(complete) lattice.

1. Let  $V$  be a set.  $(V \rightarrow P, \sqsubseteq)$  is an ordered set/(complete) lattice with

$$a \sqsubseteq b \triangleq (\forall v \in V) a(v) \leq b(v)$$

2.  $(P^n, \sqsubseteq)$  is an ordered set/(complete) lattice with

$$(a_1, \dots, a_n) \sqsubseteq (b_1, \dots, b_n) \triangleq (\forall 1 \leq i \leq n) a_i \leq b_i$$

3.  $P_\perp$  denotes the ordered set  $(P \cup \{\perp\}, \sqsubseteq)$  with  $\perp \notin P$  and

$$a \sqsubseteq b \triangleq a = \perp \text{ or } a \leq b$$

4.  $P^\top$  similarly.

# Knaster-Tarski Fixpoint Theorem

Let  $L$  be a **complete** lattice and  $f: L \rightarrow L$  be **monotone (order-preserving)**. Then

$$\bigwedge \{x \in L \mid f(x) \leq x\}$$

is the **least fixpoint** of  $f$ . (The dual holds analogously.)

# Knaster-Tarski Fixpoint Theorem

Let  $L$  be a **complete** lattice and  $f: L \rightarrow L$  be **monotone (order-preserving)**. Then

$$\bigwedge \{x \in L \mid f(x) \leq x\}$$

is the **least fixpoint** of  $f$ . (The dual holds analogously.)

Proof: Let  $R := \{x \in L \mid f(x) \leq x\}$  be the set of elements of which  $f$  is **reductive** (also called the set of **post-fixpoints**).

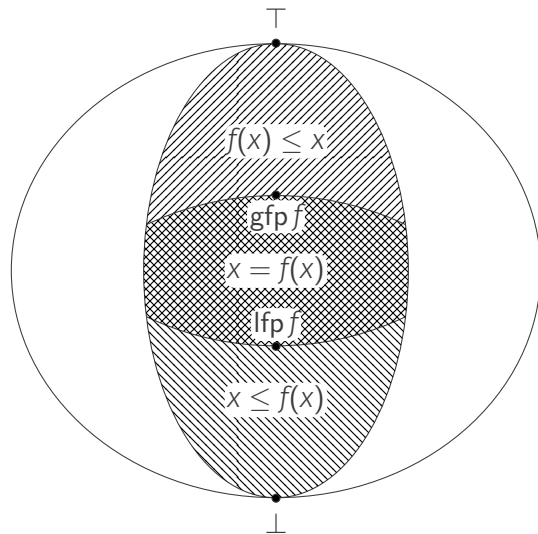
Let  $x \in R$ . Consider  $z = \bigwedge R$ .  $z$  exists because  $L$  is complete.

$z \leq x$  because  $z$  is a lower bound of  $x$ . By monotonicity,  $f(z) \leq f(x)$ . Because  $x \in R$ ,  $f(x) \leq x$ . Thus,  $f(z)$  is also a lower bound of  $R$ .

Because  $z$  is the greatest lower bound of  $R$ ,  $f(z) \leq z$ , thus  $z \in R$ . By monotonicity,  $f(f(z)) \leq f(z)$ . Hence,  $f(z) \in R$ . Because  $z$  is a lower bound of  $R$ ,  $z \leq f(z)$  and  $z = f(z)$ .



# Fixpoints in Complete Lattices



## Fixpoint by Iteration (Variation of Kleene's Theorem)

Let  $L$  be a **complete** lattice that satisfies (ACC) and  $f: L \rightarrow L$  a **monotone** function. Then, there exists a  $k$  such that  $f^k(\perp)$  is the **least fixpoint** of  $f$ .

## Fixpoint by Iteration (Variation of Kleene's Theorem)

Let  $L$  be a **complete** lattice that satisfies (ACC) and  $f: L \rightarrow L$  a **monotone** function. Then, there exists a  $k$  such that  $f^k(\perp)$  is the **least fixpoint** of  $f$ .

Proof: Let  $\beta$  be the least fixpoint of  $f$ . It exists, because of Knaster-Tarski. Because  $\perp \leq \beta$  and  $f$  is monotone,  $f^i(\perp) \leq \beta$  for all  $i \in \mathbb{N}$ .

Because

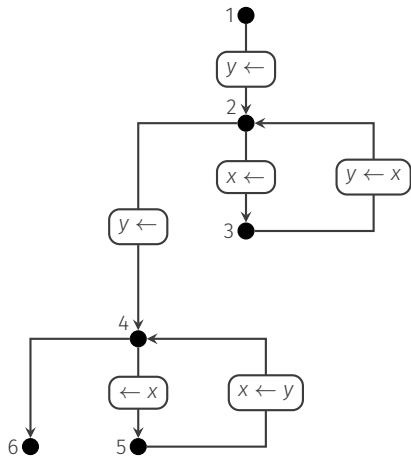
$$\perp, f(\perp), f(f(\perp)), \dots, f^i(\perp), \dots$$

is an ascending chain (follows by induction from monotonicity of  $f$ ) and  $L$  fulfills ACC, there exists a  $k$  such that  $f^k(\perp) = f^{k+1}(\perp)$ .

## How can we apply this to program analysis?

- We require the analysis domain to be a **complete lattice** with ACC.
  - For liveness and definite assignment, we had  $V \rightarrow \mathcal{P}(\text{Vars})$  which is a complete lattice with ACC, because  $\mathcal{P}(\text{Vars})$  is a finite lattice.
- We require every local edge transformer  $f_{uv}$  to be **monotone**.  
Then, the “big” function  $F$  for the entire CFG is **monotone**.
- We compute  $F$ ’s least fixpoint using Kleene’s theorem
- And have a solution for our inequation!

## The Example Again



## “Chaotic” Iteration

If  $F$  is the best (most precise) solution to our analysis problem and we can compute it by computing the finite chain

$$\perp, F(\perp), F(F(\perp)), \dots$$

This is usually inefficient for program analysis because some components don't change

We can also reach this fixpoint by updating only single program points per step in **any order** as long as we make progress!  
(see exercise sheet)

# The Worklist Algorithm to Compute the Least Fixpoint

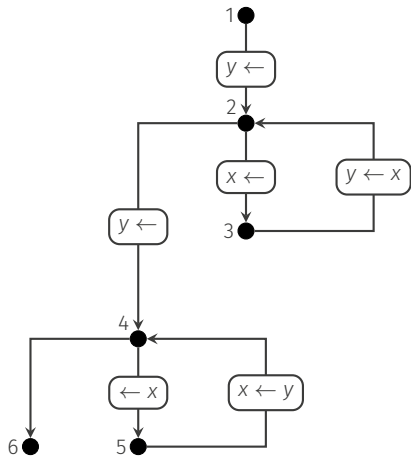
Chaotic iteration justifies the worklist algorithm that evaluates an edge transformer only when the analysis value at some program point changed.

Assume we have a domain  $V \rightarrow \mathbb{L}$  with some complete lattice  $\mathbb{L}$  and edge transformers  $f_{uv} : \mathbb{L} \rightarrow \mathbb{L}$ .

1. Initialize analysis information  $\mathbf{L}$  to  $\lambda v. \perp$   
Initialize a queue of program points  $Q$  to contain the root of the CFG
2. While  $Q$  not empty  
Pop first program point  $v$  from queue  
For each program point  $w$  with  $(v, w) \in E$   
If  $f_{vw}(\mathbf{L}(v)) \not\sqsubseteq \mathbf{L}(w)$   
Update  $\mathbf{L}(w) := \mathbf{L}(w) \sqcup f_{vw}(\mathbf{L}(v))$   
Add  $w$  to  $Q$  if not already contained

**Note:** If edge transformers push information in reverse order (e.g. liveness), initialize with end node and iterate over predecessors!

## The Example Again





# Constant Propagation

---

## Goal

Eliminate computations on variables that are constant.

## Definition

A variable  $x$  is constant with value  $v$  at program point  $p$  if in every execution  $x$  has value  $v$  at  $p$ . (more formal definition later.)

## Soundness

By Abstract Interpretation (later).

# Constant Propagation

- Domain:  $V \rightarrow (Vars \rightarrow \mathbb{Z}_{\perp}^T)$ :
  - Per program point a mapping from variables to constant information
  - $\top$ : Not constant,  $\perp$ : Not assigned

Functions on edges ( $cp_{uv}$ ):

$$\llbracket x \leftarrow e \rrbracket_{cp}^{\#} C \triangleq C[x \mapsto \llbracket e \rrbracket^{\#} C]$$

$$\llbracket x \leftarrow M[e] \rrbracket_{cp}^{\#} C \triangleq C[x \mapsto \top]$$

$$\llbracket M[e_1] \leftarrow e_2 \rrbracket_{cp}^{\#} C \triangleq C$$

Abstract expression evaluation:

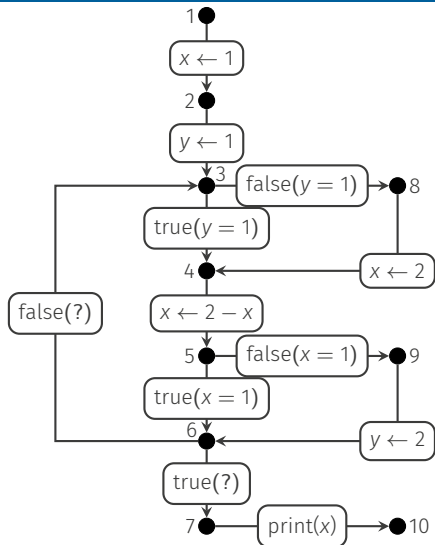
$$\llbracket e_1 \oplus e_2 \rrbracket^{\#} C \triangleq \llbracket e_1 \rrbracket^{\#} C \oplus^{\#} e_2 C$$

$$\llbracket c \rrbracket^{\#} C \triangleq c$$

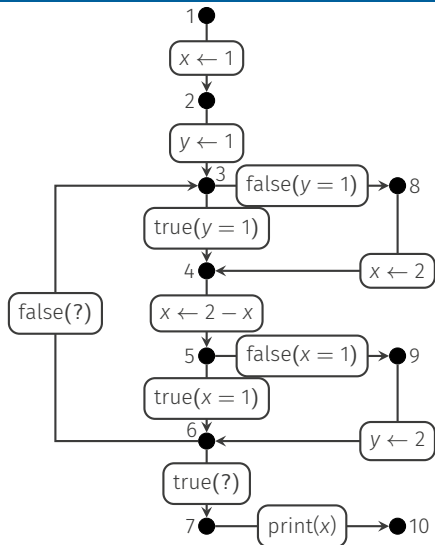
$$\llbracket x \rrbracket^{\#} C \triangleq Cx$$

$$a \oplus^{\#} b \triangleq \begin{cases} \top & \text{if } a = \top \text{ or } b = \top \\ a \oplus b & \text{otherwise} \end{cases}$$

# Example



# Example



| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|
| xy | xy | xy | xy | xy | xy | xy | xy | xy |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |

# Unreachable Code

- We could not prove as much constant as we wanted to
- Reason: Join included unreachable code
- Can use unreachability information to refine constantness and constantness to refine unreachability!
- Additional domain:  $V \rightarrow \{\perp_U, \top_U\}$ . ( $\perp_U$ : unreachable,  $\top_U$ : reachable)
- Transfer functions

$$\begin{array}{l}
 \llbracket \bullet \xrightarrow{\text{true}(e)} \bullet \rrbracket_{ur}^\# C, U \triangleq U \left[ v \mapsto \begin{cases} \perp_U & \llbracket e \rrbracket^\# C \subseteq 0 \\ \top_U & \text{otherwise} \end{cases} \right] \\
 \llbracket \bullet \xrightarrow{\text{false}(e)} \bullet \rrbracket_{ur}^\# C, U \triangleq U \left[ v \mapsto \begin{cases} \top_U & 0 \subseteq \llbracket e \rrbracket^\# C \\ \perp_U & \text{otherwise} \end{cases} \right] \\
 \llbracket \bullet \xrightarrow{\square} \bullet \rrbracket_{ur}^\# C, U \triangleq U
 \end{array}$$

## Domain Cooperation: Unreachable Code, Constant Propagation

- When joining over all incoming abstract values in constant propagation ignore predecessors that are unreachable

$$CP(C, U) \triangleq \lambda w. \bigsqcup_{v \in \text{pred}(w) \text{ and } U(v) = \top_U} cp_{vw}(C, U)$$

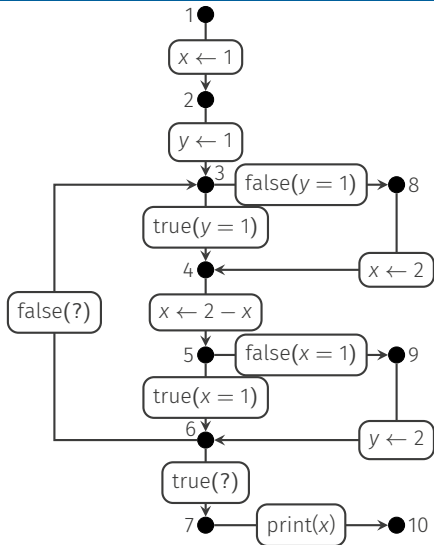
- We essentially have two inequations:

$$C \sqsupseteq CP(C, U)$$

$$U \sqsupseteq UR(C, U)$$

- But again, we can combine them into a “big” function and apply the fixpoint theorem to get a solution

# Example



| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|
| xy | xy | xy | xy | xy | xy | xy | xy | xy |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |
|    |    |    |    |    |    |    |    |    |



# Abstract Interpretation

---

# Abstract Interpretation

- (Some) analysis domains can be interpreted as **abstractions** of concrete program properties
- For example: In **constant propagation**, an element of  $Vars \rightarrow \mathbb{Z}_{\perp}^T$  can be seen as an abstraction of a set of program states
- A **state** is a (partial) mapping from variables to values  $\Sigma : Vars \rightarrow \mathbb{Z}$  and exists in **concrete** program executions
- Example:  $\{x \mapsto \top, y \mapsto 5, z \mapsto \perp\} \in Vars \rightarrow \mathbb{Z}_{\perp}^T$  represents all states  $\{\sigma \mid \exists v. \sigma x = v \wedge \sigma y = 5\}$
- We can use this to prove program analyses correct with respect to the program's semantics

## Other Common Abstract Domains

## Concretization (Constant Propagation)

- Let  $\mathbb{S} \triangleq (\mathcal{P}(\Sigma), \subseteq)$  be the complete lattice of state sets with subset inclusion
- Let  $\mathbb{C} \triangleq (Vars \rightarrow \mathbb{Z}_{\perp}^T, \sqsubseteq)$  be the constant propagation lattice (as previously defined)
- We define what an abstract element **represents** by defining a **concretization function**  $\gamma_{\mathbb{S}} : \mathbb{C} \rightarrow \mathbb{S}$ :

$$\gamma_{\mathbb{S}}(C) \triangleq \{\sigma \mid \sigma x \sqsubseteq Cx\}$$

(Note that  $\sqsubseteq$  is here the order of the lattice  $\mathbb{Z}_{\perp}^T$ .)

# Correctness of the Edge Transformers

- We can use  $\gamma$  to define the correctness criterion of the edge transformers
- Each control flow edge has a **concrete** edge effect  $f_{uv} : \Sigma \rightarrow \Sigma$ , i.e. the instruction on the edge
- Each program analysis comes with
  - An analysis domain  $\mathbb{A}$  (e.g. in const prop  $\mathbb{A} = \text{Vars} \rightarrow \mathbb{Z}_{\perp}^{\top}$ )
  - An **abstract** edge effect  $f_{uv}^{\#} : \mathbb{A} \rightarrow \mathbb{A}$  for each concrete effect  $f_{uv}$
- An edge abstract transformer  $f_{uv}^{\#}$  is **correct**, if

$$(f_{uv} \circ \gamma)(a) \subseteq (\gamma \circ f_{uv}^{\#})(a) \quad \text{for all } a \in \mathbb{A}$$

- A correct  $f_{uv}^{\#}$  **over-approximates** the concrete effect
- How do we leverage this to show the correctness of the entire analysis?

# Correctness of the Edge Transformers

# Finite Prefix Trace Semantics

- A **configuration**  $Conf \triangleq V \times \Sigma$  is a pair of a program point and a state
- A **trace**  $t$  is a sequence of configurations:  $t \in Conf^*$
- The **finite prefix trace semantics** is the set of all program executions
- It is the **least fixpoint** of the following (monotone) function:

$$F(T) \triangleq \{(r, \sigma) \mid \sigma \in \Sigma\} \\ \cup \{\langle c_0, \dots, c_n, c_{n+1} \rangle \mid \langle c_0, \dots, c_n \rangle \in T \wedge c_n \rightarrow c_{n+1}\}$$

with

$$(p, \sigma) \rightarrow (q, \sigma') \triangleq \sigma' = f_{pq}(\sigma)$$

- Define the concretization of const prop  $V \rightarrow (Vars \rightarrow \mathbb{Z}_{\perp}^T)$  to traces

$$\gamma(C) \triangleq \{t \in Conf^* \mid (p, \sigma) \in t \implies \sigma \in \gamma_S(Cp)\}$$





## Theorem

Given a monotone function  $\gamma : \mathbb{A} \rightarrow \mathbb{C}$  and two monotone functions  $F : \mathbb{C} \rightarrow \mathbb{C}$  and  $F^\sharp : \mathbb{A} \rightarrow \mathbb{A}$  for which holds  $F \circ \gamma \subseteq \gamma \circ F^\sharp$  then

$$\text{lfp } F \subseteq \gamma(\text{lfp}(F^\sharp))$$

## Proof.

By assumption,  $(F \circ \gamma)(\text{lfp } F^\sharp) \subseteq (\gamma \circ F^\sharp)(\text{lfp } F^\sharp)$ . Hence,  $F(\gamma(\text{lfp } F^\sharp)) \subseteq \gamma(\text{lfp } F^\sharp)$ . By the fixpoint theorem of Kanster-Tarski, the least fixpoint of  $F$  is less than each  $x$  with  $F(x) \subseteq x$ . Hence,  $\text{lfp } F \subseteq \gamma(\text{lfp } F^\sharp)$ .  $\square$

## Conclusion for Program Analysis

If  $F^\sharp$  is sound wrt to  $F$  then the final analysis result is a correct over-approximation of the program semantics

# Best Abstractions

Let  $(\mathbb{A}, \sqsubseteq)$  and  $(\mathbb{C}, \subseteq)$  be complete lattices and let  $\gamma : \mathbb{A} \rightarrow \mathbb{C}$  a monotone function.

## Theorem

$\gamma$  and the function  $\alpha(c) \triangleq \bigsqcap \{a \mid c \subseteq \gamma(a)\}$  form a *Galois connection*, i.e.

$$c \subseteq \gamma(a) \iff \alpha(c) \sqsubseteq a$$

## What does this mean?

$\alpha$  returns the *best* abstraction for a given  $c \in \mathbb{C}$ .

## Best Abstract Transformer

The (correct) *best abstract transformer* is given by  $\alpha \circ F \circ \gamma$ .

(it is sometimes interesting to show that one's transformer is actually the best.  
This means that we do not lose precision with respect to the analysis domain.)

# Summary

To design a program analysis, you need:

- A **complete lattice** that represents your analysis domain
- **Monotone abstract transformers**  $f_{uv}^\#$  for each edge of the CFG

By the presented framework, you get:

- The monotone abstract transformer  $F^\#$  (e.g. LV, DA, CP)
- An analysis algorithm that computes the **least fixpoint** of  $F^\#$

To prove its **correctness**, you need to provide:

- A **program semantics** by concrete edge transformers  $f_{uv}$
- **Correctness proofs** of the edge transformers  $f \circ \gamma \subseteq \gamma \circ f^\#$

By the presented framework, you get:

- that the analysis correctly over-approximates the program's behavior

## Pentagons and Bound Checking

---

# Motivation

```
int binarySearch(int[] a, int value) {  
    int l = 0, u = a.length - 1;  
    while (l <= u) {  
        int i = (l + u) / 2;  
        int v = a[i];  
        if (v == value) return i;  
        if (v < value) l = i + 1;  
        else          u = i - 1;  
    }  
    return ~l;  
}
```

Java requires to throw an exception if the array access is out of bounds.

# Motivation

So the code that is really executed is:

```
int binarySearch(int[] a, int value) {  
    int l = 0, u = a.length - 1;  
    while (l <= u) {  
        int i = (l + u) / 2;  
        int v;  
        if (i < 0 || i >= a.length) throw new ...  
        else v = a[i];  
        if (v == value) return i;  
        if (v < value) l = i + 1;  
        else          u = i - 1;  
    }  
    return ~l;  
}
```

- Apparently, the condition is always true and the compiler should eliminate the bounds check and remove the throw.
- With interval analysis we only get the bound  $i \in [0, \infty]$
- Not powerful enough to provide relational information  $i < a.length$

## Strict Upper Bounds Domain (sub)

- Represent strict inequalities, like  $x < y$
- Domain:  $Vars \rightarrow \mathcal{P}(Vars)$   
Map each  $x$  to all variables that are strictly greater than  $x$
- Concretization:  $\gamma_{\text{sub}} : s \mapsto \{\text{state } \sigma \mid \forall xy : y \in s(x) \Rightarrow \sigma(x) < \sigma(y)\}$

## Strict Upper Bounds Domain (sub)

- Represent strict inequalities, like  $x < y$
- Domain:  $Vars \rightarrow \mathcal{P}(Vars)$   
Map each  $x$  to all variables that are strictly greater than  $x$
- Concretization:  $\gamma_{\text{sub}} : s \mapsto \{\text{state } \sigma \mid \forall xy : y \in s(x) \Rightarrow \sigma(x) < \sigma(y)\}$
- Join:  $s \sqcup_{\text{sub}} t : \iff \lambda x. (s(x) \cap t(x))$   
implies ordering via  $a \sqsubseteq_{\text{sub}} b \iff a \sqcup_{\text{sub}} b = b$
- $\top = \lambda x. \emptyset$  and  $\perp = \lambda x. Vars$



# Closures

- Because  $<$  is transitive, there are many elements in sub that concretize to the same set of states, e.g. consider

$$s_1 = [x \mapsto \{y\}, y \mapsto \{z\}]$$

$$s_2 = [x \mapsto \{y, z\}, y \mapsto \{z\}]$$

for which we have  $\gamma(s_1) = \gamma(s_2)$

- When joining, it actually makes a difference which one we have:

$$s_1 \sqcup [x \mapsto \{z\}] = \top$$

$$s_2 \sqcup [x \mapsto \{z\}] = [x \mapsto \{z\}]$$

- Because  $\gamma_{\text{sub}}$  is monotone, there is a **best abstraction**  $\alpha(c)$
- Therefore,  $(\alpha \circ \gamma)(s)$  gives the best abstraction for  $\gamma(s)$  (i.e. the best abstraction representing the same concrete element as  $s$ )
- $(\alpha \circ \gamma)$  is called a **closure operator**

- To make the join most precise one could compute the **closure**  $\alpha \circ \gamma$  and join with the best abstractions
- The closure operator can in practice be expensive:  
In **sub** one has to compute the transitive closure of the relation represented by an abstract element
- In practice other operations that overapproximate the join are imaginable.

- Using sub **without** intervals does not help in proving the array access in bounds in our example. Information about constants missing
- Hence: Use both analyses:  $\text{pentagons} = \text{sub} \times \text{intervals}$

## Reduced Product

- In the product, there are typically multiple abstract elements that are concretized to the same value:

$$\begin{aligned} & \gamma(\langle \{x \mapsto [0, 100], y \mapsto [0, 50]\}, \{x < y\} \rangle) \\ = & \gamma(\langle \{x \mapsto [0, 49], y \mapsto [1, 50]\}, \{x < y\} \rangle) \end{aligned}$$

- Therefore, one also gets a closure operator that gives the best abstraction in  $\text{sub} \times$  intervals for a given abstraction:

$$\langle b, s \rangle \mapsto \langle b^*, s^* \rangle$$

$$b^* = \bigcap_{\{x < y\} \in s} \llbracket x < y \rrbracket^\#(b)$$

$$s^* = \lambda x. s(x) \cup \{y \in \text{Vars} \mid x^u < y^\ell\} \quad \text{with } b(z) = [z^\ell, z^u]$$

- Applying the closure operator might be expensive.  
In Pentagons  $O(|Vars|^2)$
- To get the best precision, one has to do it before every operation:  
join, application of abstract transformer.
- Hence, in practice, one uses
  - A less precise but more efficient join,  
e.g. in Pentagons, ignore sub information for interval join.
  - Modified abstract transformers, that integrate information from both domains,  
intervals and sub. For example, consider subtraction with:

$$\llbracket \mathbf{r} \leftarrow \mathbf{x} - \mathbf{y} \rrbracket^\sharp \langle b, s \rangle = \langle b[\mathbf{r} \mapsto b_r], s[\mathbf{r} \mapsto s_r] \rangle \quad \text{with}$$

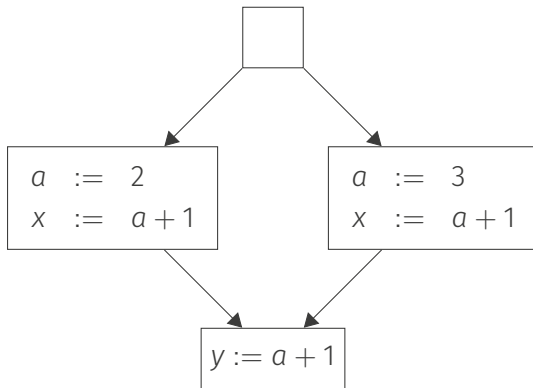
$$b_r = \llbracket \mathbf{x} - \mathbf{y} \rrbracket_{\text{intv}}^\sharp \cap ((\mathbf{y} < \mathbf{x}) \in s ? [1, \infty] : \top_{\text{intv}})$$

$$s_r = \mathbf{y}^\ell > 0 ? \{\mathbf{x}\} \cup s(\mathbf{x}) : \emptyset$$

## Global Value Numbering

---

# Value Numbering



- Replace second computation of  $a + 1$  with a copy from  $x$

- Goal: Eliminate redundant computations
- Find out if two variables have the same value at given program point
  - In general undecidable
- Potentially replace computation of latter variable with contents of the former
- Resort to **Herbrand equivalence**:
  - Do not consider the interpretation of operators
  - Two expressions are equal if they are structurally equal
- This lecture: A costly program analysis which finds all Herbrand equivalences in a program and a “light-weight” version that is often used in practice.



# Herbrand Interpretation

- Let  $T$  be the set of terms over a set of variables
- The Herbrand interpretation  $\mathcal{I}$  of an  $n$ -ary operator  $\omega$  is given as

$$\mathcal{I}(\omega) : T^n \rightarrow T \quad \mathcal{I}(\omega)(t_1, \dots, t_n) := \omega(t_1, \dots, t_n)$$

Treat operators as **uninterpreted functions**

- With a state  $\sigma$  that maps variables to terms

$$\sigma : Vars \rightarrow T$$

- Define the **Herbrand semantics**  $\langle t \rangle \sigma$  of a term  $t$

$$\langle t \rangle \sigma := \begin{cases} \sigma(v) & \text{if } t = v \text{ is a variable} \\ \mathcal{I}(\omega)(\langle x_1 \rangle \sigma, \dots, \langle x_n \rangle \sigma) & \text{if } t = \omega(x_1, \dots, x_n) \end{cases}$$

- Two terms  $t, t'$  are **Herbrand-equivalent**, if for all states  $\sigma$ :  $\langle t \rangle \sigma = \langle t' \rangle \sigma$

# Kildall's Analysis

- Track Herbrand equivalences with a **forward** analysis
- A lattice element is a partition of the terms **and** variables
- The equivalence relation is a **congruence relation** w.r.t. to the operators in our expression language.

For each operator  $\omega$ , each eq. relation  $R$ , and  $e, e_1, \dots \in \text{Vars} \cup T$ :

$$e_1 R e'_1 \implies e_2 R e'_2 \implies (e_1 \omega e_2) R (e'_1 \omega e'_2)$$

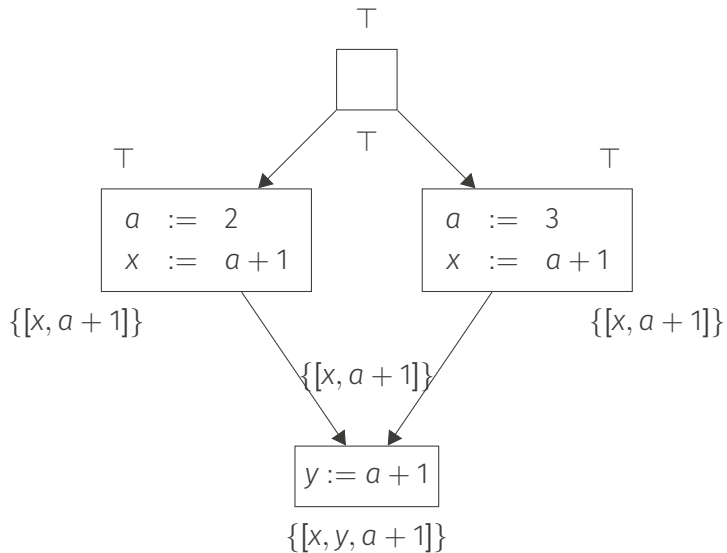
(Kildall calls this **structured** partition)

- Two partitions are joined by intersecting them

$$R \sqcup S := R \cap S := \{(a, b) \mid a R b \wedge a S b\}$$

- $\perp = \{(x, y) \mid x, y \in \text{Vars} \cup T\}$
- $\top = \{(x, x) \mid x \in \text{Vars} \cup T\}$

# Kildall's Analysis

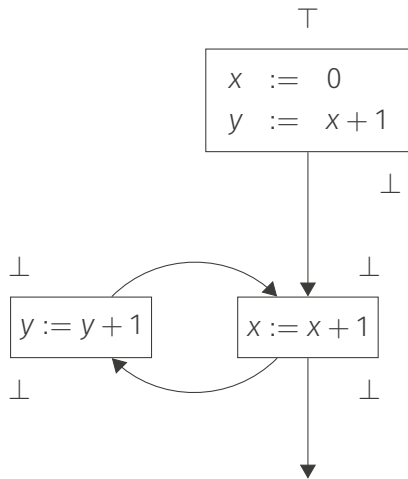


Abstract transformer of an assignment  $x \leftarrow t$ :

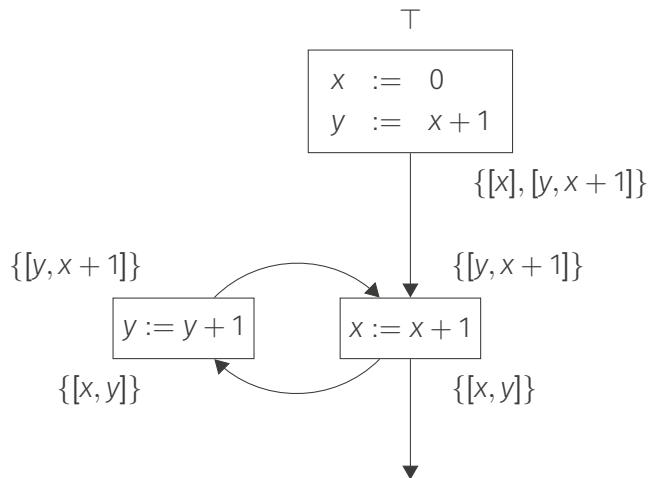
- Compute a new partition by checking (in the old partition) who is equivalent if we replace  $x$  by  $t$

$$\llbracket x \leftarrow t \rrbracket^\# R := \{(t_1, t_2) \mid t_1[t/x] R t_2[t/x]\}$$

# Kildall's Analysis



# Kildall's Analysis



## Kildall's Analysis / Some Comments

- Kildall's Analysis is **sound** and **complete** (w.r.t. to Herbrand Semantics): it discovers **all** Herbrand equivalences in the program
- Straight-forward implementation exhibit pathological exp. explosion:
  - Because the equivalence relation must be a congruence, the size of equivalence classes can **theoretically** explode:

$$R = \{[a, b], [c, d], [e, f], [x, a + c, a + d, b + c, b + d], \\ [y, x + e, x + f, (a + c) + e, \dots, (b + d) + f]\}$$

- Use better data structure to store the congruence:  
**Value graph** (Gulwani & Necula, Rüthing et al.):
  - Even in acyclic programs, detecting all equivalences can lead to exponential-sized value graphs
  - Detecting only equivalences among terms in the program is polynomial (linear-sized representation of equivalences per program point)

# Strong Equivalence DAGs (SED)

A **SED**  $G$  is a DAG  $(N, E)$ . Let  $N$  be the set of nodes of the graph. Every node  $n$  is a pair  $(V, t)$  of a set of variables and a type<sup>1</sup>

$$t ::= \perp \mid c \mid \oplus(n_1, \dots, n_k)$$

A type  $\oplus(n_1, \dots, n_k)$  indicates, that

$$\{(n, n_1), \dots, (n, n_k)\} \in E$$

A node  $n = (V, t)$  in the SED stands for a set of terms  $T(n)$

$$T((V, \perp)) \triangleq V$$

$$T((V, c)) \triangleq V \cup \{c\}$$

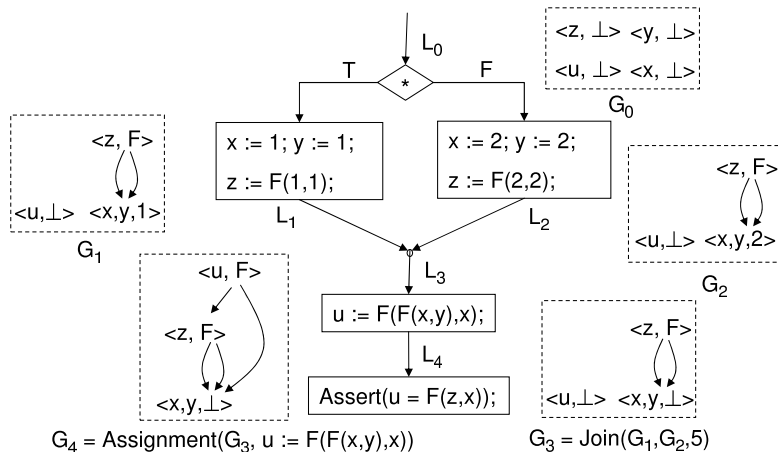
$$T((V, \oplus(n_1, \dots, n_k))) \triangleq V \cup \{\oplus(e_1, \dots, e_k) \mid e_i \in T(n_i)\}$$

---

<sup>1</sup>Note that  $\perp$  does not denote the “empty set of states” here



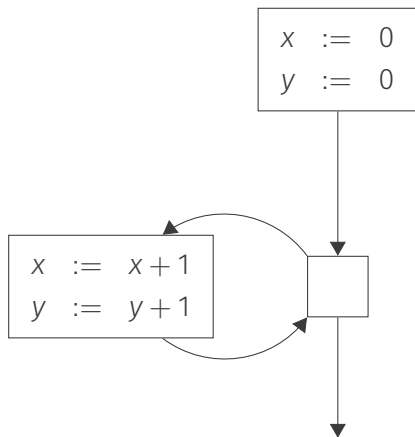
# Strong Equivalence DAGs (SED)



# The Alpern, Wegman, Zadeck (AWZ) Algorithm

- Incomplete
- Flow-insensitive
  - does not compute the equivalences for every program point but sound equivalences for the whole program
- Uses SSA
  - Control-flow joins are represented by  $\phi$ s
  - Treat  $\phi$ s like every other operator (cause for incompleteness)
  - Source of imprecision
- Interpret the SSA data dependence graph as a finite automaton and minimize it
  - Refine partitions of “equivalent states”
  - Using Hopcroft’s algorithm, this can be done in  $O(e \cdot \log e)$

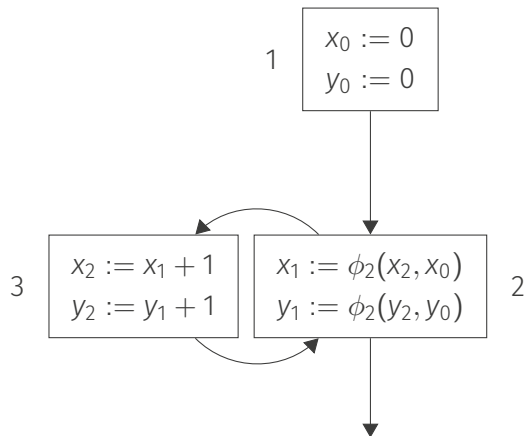
# The AWZ Algorithm



# The AWZ Algorithm

- In contrast to finite automata, do not create two partitions but a class for every operator symbol
  - Note that the  $\phi$ 's block is part of the operator
  - Two  $\phi$ s from different blocks have to be in different classes
- Optimistically place all nodes with the same operator symbol in the same class
  - Finds the least fixpoint
  - You can also start with singleton classes and merge but this will (in general) not give the least fixpoint
- Successively split class when two nodes in the class are detected not equivalent

# The AWZ Algorithm



## Kildall compared to AWZ

